

게임서버 템프 설명서

2022182002 강지호

1. 네트워크 통신 프로토콜 (Protocol)

서버는 패킷의 첫 1바이트 크기(size)와 다음 1바이트 타입(type)을 읽어 패킷을 해석하는 가변 길이 패킷 구조를 채택하고 있으며, pragma pack(push, 1) 구조체 정렬을 통해 패킷 데이터 패딩을 제거하고 통신 효율을 극대화했습니다. 아래는 코드에 구현된 모든 프로토콜 일람입니다.

1) 클라이언트 → 서버 패킷 (C2S)

- **C2S_LOGIN**: 플레이어가 로그인을 위해 사용자 이름(username)을 전송하는 패킷입니다. 서버는 이 이름을 받아 데이터베이스에 조회를 요청합니다.
- **C2S_MOVE**: 플레이어의 상대 이동 좌표(x, y 변위)와 클라이언트 기준 이동 시간(move_time)을 전송합니다. 서버는 이 패킷을 기반으로 이동 유효성 및 맵 충돌을 검사합니다.
- **C2S_CHAT**: 유저가 입력한 문자열 메시지(message)를 전달합니다.
- **C2S_ATTACK**: 현재 장착한 무기로 주변을 타격하겠다는 공격 요청 패킷입니다. 별도의 타겟 ID 없이 플레이어 중심의 범위 충돌 판정 알고리즘을 발생시킵니다.
- **C2S_TELEPORT**: 스트레스 테스트 및 시작 지점 밀집 방지를 위해 목적지 절대 좌표(x, y)를 강제로 요청하는 패킷입니다.
- **C2S_SELECT_WEAPON**: 검(sword), 망치(hammer), 창(spear) 중 하나를 선택하여 자신의 무기 상태(weapon)를 변경 및 등록합니다.
- **C2S_REQUEST_INFO**: 타겟 유저명(target_username)을 기반으로 해당 캐릭터의 무기, 체력, 강화 단계 등의 상세 정보를 동기화해달라고 서버에 요청합니다.
- **C2S_DUNGEON_ENTRY**: 특정 던전 타입(dungeon) 입장을 요청합니다. 포탈 배치 정보를 기반으로 현재 플레이어 좌표 및 레벨 제한을 검사받습니다.
- **C2S_INTERACT**: 상인 NPC 주변 1칸 이내에서 상호작용을 시도할 때 전송하며, 다음 강화에 필요한 비용 등의 가변 정보를 획득하는 데 사용됩니다.
- **C2S_REINFORCE**: 현재 무기와 강화 레벨 정보, 보유 골드를 기반으로 강화 상인에게 무기 업그레이드를 요청하는 패킷입니다.

- **C2S_INVITE_PARTY / C2S_ACCEPT_PARTY / C2S_REFUSE_PARTY:** 특정 대상 유저에게 파티 초대를 발송하거나, 초대받은 요청에 대해 수락 및 거절 의사를 전달합니다.
- **C2S_LEAVE_PARTY:** 현재 소속되어 있는 파티 그룹에서 이탈을 선언합니다.
- **C2S_LOGOUT:** 정상적인 종료 흐름을 밟아 세션을 안전하게 닫고 현재의 최신 캐릭터 정보를 DB에 즉시 동기화하도록 유도합니다.

2) 서버 → 클라이언트 패킷 (S2C)

- **S2C_LOGIN_RESULT:** 로그인 결과 성공 여부(success)와 실패 사유 또는 안내 메시지(message)를 클라이언트에 회신합니다.
- **S2C_AVATAR_INFO:** 로그인에 성공한 본인 캐릭터의 고유 ID, 이름, 스폰 좌표, 스탯(HP, MAX_HP, EXP, LEVEL)을 전송하여 내 화면을 초기화합니다.
- **S2C_ADD_OBJECT:** 플레이어의 시야(Grid Sector) 내로 새롭게 진입한 다른 플레이어 혹은 NPC/몬스터 객체의 모든 정보와 외형 ID(visual_id)를 전달하여 화면에 스폰시킵니다.
- **S2C_REMOVE_OBJECT:** 시야에서 벗어나거나 사망하여 사라진 객체의 ID(object_id)를 전송해 클라이언트 렌더링에서 삭제되도록 만듭니다.
- **S2C_MOVE_OBJECT:** 동기화 범위 내 특정 객체가 이동했을 때 해당 객체의 ID와 새로운 목적지 좌표, 이동 시간을 공유해 클라이언트 화면에서 부드러운 이동을 연출하도록 지원합니다.
- **S2C_CHAT_MESSAGE:** 시야 범위 안에 존재하는 다른 유저들이 입력한 일반 대화 텍스트나 시스템 메시지를 전달합니다.
- **S2C_STATUS_CHANGE:** 전투, 레벨업, 회복 등으로 인해 대상 객체의 HP, MaxHP, 경험치, 레벨 스탯이 실시간 변동될 때 이를 광범위하게 동기화합니다.
- **S2C_DUNGEON_RESULT:** 던전 이동 결과 처리에 대한 피드백 패킷으로, 성공 시 실제 이동할 인스턴스 던전 내부 좌표를 넘겨줍니다.
- **S2C_INFO_RESULT:** 정보 요청의 응답으로 대상 유저의 장착 무기 상태, 강화 수치, 소지 골드, 파티 가입 여부 등의 복합 데이터를 반환합니다.
- **S2C_INTERACT_RESULT / S2C_REINFORCE_RESULT:** NPC 상호작용 및 강화 연산 결과(성공/실패 여부, 변동된 골드, 최종 강화 차수)를 동기화합니다.
- **S2C_PARTY_INVITE_NOTI:** 다른 플레이어로부터 파티 초대장을 받았을 때 초대자

의 ID와 이름 정보를 담아 화면에 수락/거절 팝업을 띄우도록 클라이언트에 노트를 줍니다.

- **S2C_PARTY_UPDATE**: 파티 멤버 가입/탈퇴/능력치 변동 시 최대 4인의 멤버 배열 데이터(PartyMemberInfo) 전체를 브로드캐스트하여 파티창 UI를 최신화합니다.
- **S2C_ATTACK_BROADCAST**: 다른 유저가 공격 패턴을 발동했을 때 무기 종류 정보를 포함해 공격 이펙트를 사방에 연출하도록 명령합니다.
- **S2C_GOLD_UPDATE**: 몬스터 처치 및 무기 강화 성공/실패 시 소지 골드 보유액 정보를 갱신합니다.
- **S2C_BOSS_WARN_ZONE**: 최종 보스가 광역기 캐스팅 시 붉은색 범위 예고 이펙트를 띄울 수 있도록 중심 좌표와 반경(radius), 제한 시간(duration_ms) 정보를 내려줍니다.
- **S2C_STATUS_EFFECT**: 보스의 대지분쇄 타격에 맞았을 때 플레이어 캐릭터가 is_confused 상태(조작 반전 상태 효과)에 돌입했거나 해제되었음을 클라이언트에 통보합니다.

2. 게임 구조를 지탱하는 핵심 자료구조 (Data Structure) 분석

대규모 멀티스레드 환경에서 데이터 경합(Race Condition)을 방지하고 메모리 병목 현상을 타파하기 위해 최적화된 자료구조 조합이 사용되었습니다.

1) 공간 분할 격자: Sector 배열

- **코드 내 타입**: `Sector g_sectors[SECTOR_COUNT_Y][SECTOR_COUNT_X];`
- **구현 명세**: 각 Sector 구조체 내부에는 스레드 안전한 해시맵인 `tbb::concurrent_unordered_map<int, int> players;`가 포함되어 있습니다.
- **의미 및 사용 목적**: \$2000 \times 2000\$ 크기의 방대한 월드 맵에서 주변 플레이어를 찾기 위해 매번 모든 접속 유저(최대 10,000명)를 순회하는 것은 연산 비용이 너무 큼니다. 가시거리 범위인 7칸 단위를 기준으로 맵을 쪼개어 격자형 섹터를 구성하고, 플레이어가 이동할 때마다 자신이 속한 섹터 해시맵에 오브젝트 ID를 등록/삭제하는 방식으로 주변 탐색 범위를 제한하여 CPU 연산 효율을 높입니다.

2) 글로벌 객체 보관소: g_objects 및 g_sessions

- **코드 내 타입**:
 - `tbb::concurrent_unordered_map<int,`

`std::atomic<std::shared_ptr<GameObject>>> g_objects;`

- `tbb::concurrent_unordered_map<int, std::atomic<std::shared_ptr<Session>>> g_sessions;`

- **의미 및 사용 목적:** 게임 내의 모든 플레이어와 20만 마리의 NPC/몬스터 객체들을 통합 관리하는 고성능 보관소입니다. 멀티스레드가 동시에 특정 객체를 삭제하거나 수정할 때 메모리 파편화 및 덩글링 포인터오류가 나는 것을 막기 위해 인텔 TBB의 동시성 맵 구조에 스마트 포인터를 원자적으로 적재하여 Lock-Free에 가까운 안전한 메모리 생명주기 관리를 달성했습니다.

3) 정렬형 시각 스케줄러: `timer_queues`

- **코드 내 타입:** `concurrency::concurrent_priority_queue<event_type> timer_queues[NUM_TIMER_QUEUES];`
- **의미 및 사용 목적:** 몬스터의 로밍 이동, 보스의 스킬 폭발, 혼란 상태 이상 해제, 사망 후 30초 뒤 리스폰 등 시간차로 발생하는 비동기 이벤트를 순차 제어하는 스케줄러입니다. 이벤트 발생 예정 시각(`wakeup_time`)을 오버로딩하여 오름차순으로 자동 정렬되도록 커스텀 힙 구조를 갖추었으며, 락 컨텐션을 분산시키기 위해 ID 기반으로 총 16개의 독립된 우선순위 큐 배열로 쪼개어 관리합니다.

4) 파티 동기화 그룹체: `Party` 와 `g_parties`

- **코드 내 타입:** `std::unordered_map<int, std::shared_ptr<Party>> g_parties;`
- **구현 명세:** 구조체 내부에 `int leader_id;`, `std::vector<int> members;`, 그리고 그룹 내부 연산용 동기화 락 객체인 `std::mutex p_lock;`을 독립 소유합니다.
- **의미 및 사용 목적:** 최대 4인의 플레이어를 하나의 전투 부대로 바인딩합니다. 파티 가입, 탈퇴, 리더의 던전 진입에 따른 일괄 이동 연산 시 멤버 목록(`std::vector`)이 안전하게 유지되도록 전용 뮤텍스 락(`p_lock`)을 걸어 스레드 경합 이슈를 통제합니다.

3. 핵심 게임 구동 알고리즘 (Algorithms) 디테일 분석

1) 기획서 최적화 수립형 A* 길찾기 알고리즘

몬스터가 타겟 유저를 향해 이동하거나 스폰 지점으로 귀환할 때 지형지물(벽=`g_map` 내 데이터 1)을 피해 갈 수 있도록 최단 경로를 동적으로 연산해 줍니다.

- **성능 최적화 예외 처리 (간이 추적 모드):** 타겟 유저와의 거리가 10칸을 초과할 정도로 멀리 떨어져 있을 경우, 수많은 타일 노드를 연산하는 전체 A* 탐색 루프

를 건너뛰니다. 대신 $dest_x > start_x$ 조건 등을 따지는 방향 벡터 판별 알고리즘을 사용해 유저 방향으로 단순 직선 1칸을 전진시켜 CPU 자원 소모를 엄청나게 줄여 줍니다.

2) 격자 기반 가시 영역 시야 동기화 알고리즘 (Spatial Partitioning View Sync)

유저가 이동 패킷(C2S_MOVE)을 보내 좌표가 변경되었을 때, 월드에 존재하는 모든 유저에게 정보를 뿌리는 구조가 아니라 속해 있는 섹터 경계를 추적하는 알고리즘입니다.

- **섹터 변경 판별:** 이동 전 섹터 좌표 old_sx, old_sy 와 이동 후 섹터 좌표 new_sx, new_sy 를 비교 연산합니다.
- **시야 업데이트 스캔 루프:** 만약 섹터 격자 구역이 바뀌었다면, 이동 후 섹터를 기점으로 상하좌우를 감싸는 주변 3x3 범위의 인접 섹터만을 정밀 스캔 대상으로 지정합니다.
- **시야 교집합/차집합 연산:**
 - 새로운 3x3 섹터 리스트 내의 유저 및 몬스터 ID들을 `near_list` 해시셋에 취합합니다.
 - 기존에 들고 있던 원래 시야 리스트와의 차집합 연산을 수행하여, 이번 이동으로 인해 **새롭게 시야에 들어온 대상**에게는 `S2C_ADD_OBJECT` 패킷을 상호 전송하고 가상 시야 리스트에 추가 등록합니다.
 - 반대로 **시야에서 완벽히 이탈한 대상**들에게는 즉각 `S2C_REMOVE_OBJECT` 패킷을 교환하고 내 가시 영역 풀에서 말소시킵니다.
 - 섹터 영역이 바뀌지 않고 내부 타일만 한 칸 이동한 상태라면 기존 가시 리스트의 대상자들에게 단순 `S2C_MOVE_OBJECT`만을 전송해 통신 과부하를 최소화합니다.

3) 몬스터 AI 상태 제어 알고리즘 (C++ & Lua 인터페이스 모델)

게임 서버 엔진의 하드웨어 연산 효율과 기획서의 기동성 요구 사양을 결합하기 위해 C++ 코어 엔진과 외부 스크립트 엔진을 결합한 상호 호출 모델을 채택했습니다.

4) 비동기 데이터베이스 태스크-큐(Task-Queue) 알고리즘

플레이어의 정보 로드 및 실시간 세이브 쿼리가 입출력 병목 현상(I/O Bottleneck)을 유발해 메인 로직 프레임을 끊기게 만드는 고질적 문제를 해결하기 위한 비동기 파이프라인 구조입니다.

- **생산자-소비자 패턴:** 메인 워커 스레드는 게임 루프 도중 세이브나 로그인이 필요

할 때 DB 연결 소켓을 직접 쥐지 않습니다. 대신 메모리 상에서 안전한 뮤텍스로 보호되는 task_queue 뒤쪽에 가벼운 DB_Task 구조체 객체를 집어넣는 생산자 역할만 수행하고 즉시 로직으로 복귀합니다.

- **전용 백그라운드 루프:** 시스템 초기화 시 독립적으로 분기되어 실행된 UpdateDBLoop 전용 워커 스레드가 상시 대기하다가 큐에서 태스크를 하나씩 꺼냅니다.
- **ODBC 커서 최적화:** 큐에 적재된 타입이 TASK_LOGIN이면 SQLExecDirectA를 호출해 사용자 명을 SQL 쿼리로 조회하고 정보가 없다면 디폴트 로우 데이터를 INSERT하는 알고리즘을 수행하며, TASK_SAVE인 경우 실시간 스탯을 UPDATE 쿼리로 날린 뒤 SQLCloseCursor를 호출해 핸들 누수를 원천 차단합니다. 연산이 끝나면 다시 C++ 엔진의 스폰 동기화 함수를 역호출해 본 월드에 캐릭터를 안전하게 안착시킵니다.